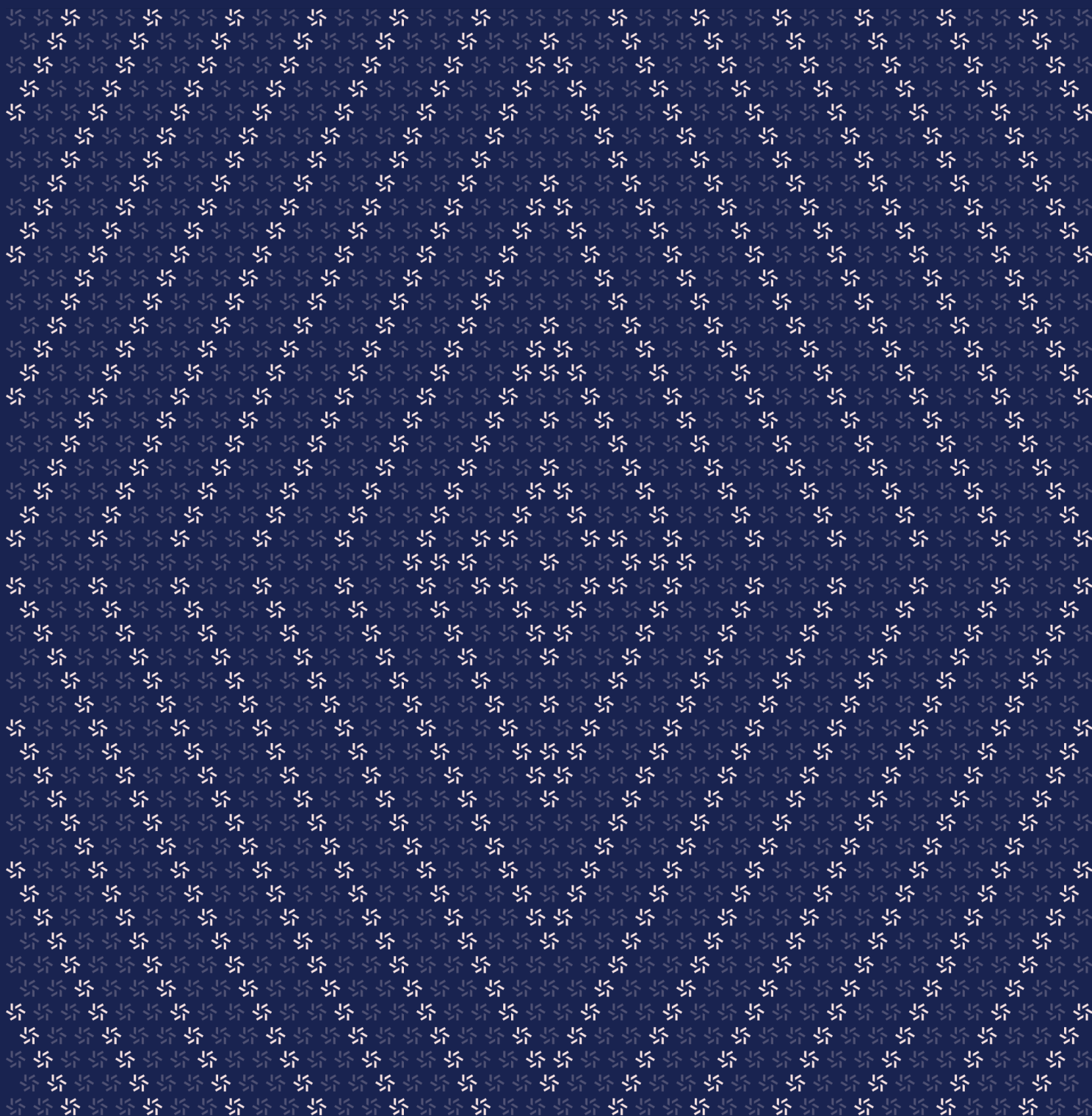


March 31, 2026

Flare FAssets V1.3

Smart Contract Patch Review



Contents

About Zellic	4
1. Overview	4
1.1. Executive Summary	5
1.2. Results	5
2. Introduction	5
2.1. About Flare FAssets V1.3	6
2.2. Methodology	6
2.3. Scope	7
2.4. Project Overview	9
2.5. Project Timeline	10
3. Detailed Findings	10
3.1. Unbounded XRP destination tag	11
3.2. The <code>_setExecutor</code> function uses the pending executor instead of effective executor, allowing cooldown bypass	13
3.3. Use of nonupgradable contract	15
3.4. Unsafe minting function may cause loss of NFTs	16
3.5. Core-vault donation-tag bypass enables griefing attack	17
3.6. Nonstandard initialization pattern	19
3.7. The <code>minimumRedeemAmountAMG</code> is not enforced by lot-based redeem	20
3.8. The <code>convertFromUSD5</code> function returns unnormalized amounts when <code>tokenFt-soSymbol</code> is empty	22

3.9.	By counting nondistributable fees, enter inflates the zero-supply entry requirement	24
3.10.	Delayed direct mints do not reserve minting cap capacity	26
3.11.	Inconsistency in DirectMintingPaymentTooSmallForFee emission	28
3.12.	Direct minting limiter behaves as a throttling mechanism rather than a strict per-window cap	30
3.13.	Missing character validation allows invalid underlying addresses in redemption requests	31
4.	Patch Review	31
4.1.	Introduction of the direct minting system	32
4.2.	Changes in contract structure	34
5.	Assessment Results	37
5.1.	Disclaimer	38

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow [@zellic_io](#) on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Flare Network from March 16th to March 25th, 2026. During this engagement, Zellic reviewed Flare FAssets V1.3's code for security vulnerabilities, design issues, and general weaknesses in security posture.

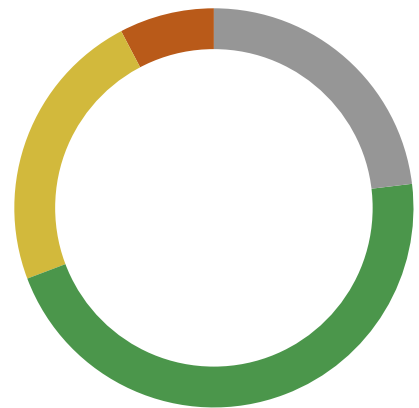
We were asked to review a minor patch to Flare FAssets V1.3, which refactored some contract structures and introduced a few new contracts, based on the previous version.

1.2. Results

During our assessment on the scoped Flare FAssets V1.3 contracts, we discovered 13 findings. No critical issues were found. One finding was of high impact, three were of medium impact, six were of low impact, and the remaining findings were informational in nature.

Breakdown of Finding Impacts

Impact Level	Count
 Critical	0
 High	1
 Medium	3
 Low	6
 Informational	3



2. Introduction

2.1. About Flare FAssets V1.3

Flare Network contributed the following description of Flare FAssets V1.3:

FAssets bring non-smart contract assets like XRP into DeFi — securely, salably, and with full custody retained.

The FAsset contracts are used to mint assets on top of Flare. The system is designed to handle chains which don't have (full) smart contract capabilities, although it can also work for smart >contract chains. Initially, FAsset system will support XRP native asset on XRPL. At a later date BTC, DOGE, add tokens from other blockchains will be added.

The minted FAssets are secured by collateral, which is in the form of ERC20 tokens on Flare/-Songbird chain and native tokens (FLR/SGB). The collateral is locked in contracts that guarantee that >minted tokens can always be redeemed for underlying assets or compensated by collateral. Underlying assets can also be transferred to Core Vault, a vault on the underlying network. When the >underlying is on the Core Vault, the agent doesn't need to back it with collateral so they can mint again or decide to withdraw this collateral.

Two novel protocols, available on Flare and Songbird blockchains, enable the FAsset system to operate:

- **FTSO** contracts which provide decentralized price feeds for multiple tokens.
- Flare's **FDC**, which bridges payment data from any connected chain.

The main update to version v1.3 is to enable the direct minting to Core Vault.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic

tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

2.3. Scope

The engagement involved a review of the following targets:

Flare FAssets V1.3 Contracts

Type	Solidity
Platform	EVM-compatible

Target	Only changes between 4c52db6...d54b04c
Repository	https://github.com/flare-foundation/fassets ↗
Version	d54b04c8fa7201f038f00121af3ef9e2d4f58cf9
Programs	contracts/agentOwnerRegistry/implementation/*.sol contracts/agentVault/implementation/*.sol contracts/assetManager/diamondInitializers/*.sol contracts/assetManager/facets/*.sol contracts/assetManager/implementation/*.sol contracts/assetManager/library/*.sol contracts/assetManager/library/data/*.sol contracts/assetManagerController/implementation/*.sol contracts/collateralPool/implementation/*.sol contracts/diamond/facets/*.sol contracts/diamond/implementation/*.sol contracts/diamond/library/*.sol contracts/fassetToken/implementation/*.sol contracts/fassetToken/library/*.sol contracts/flareSmartContracts/implementation/*.sol contracts/ftso/implementation/*.sol contracts/governance/implementation/*.sol contracts/mintingTagManager/*.sol contracts/openzeppelin/library/*.sol contracts/openzeppelin/security/*.sol contracts/userInterfaces/data/*.sol contracts/utils/library/*.sol
Target	Only changes in 62b43f7
Repository	https://github.com/flare-foundation/fassets ↗
Version	62b43f702ad16780ac76cec0d10158bc0fce0231
Programs	contracts/assetManager/facets/DirectMintingFacet.sol

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 2.4 person-weeks. The assessment was conducted by two consultants over the course of 8 calendar days.

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
✧ Engagement Manager
jacob@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Kritsada Dechawattana
✧ Engineer
kritsada@zellic.io ↗

Weipeng Lai
✧ Engineer
weipeng.lai@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

March 16, 2026 Start of primary review period

March 25, 2026 End of primary review period

3. Detailed Findings

3.1. Unbounded XRP destination tag

Target	RedemptionRequestsFacet		
Category	Coding Mistakes	Severity	High
Likelihood	Medium	Impact	High

Description

Note: This finding was discovered independently by Flare Network during the review period.

The `redeemWithTag()` function accepts a `_destinationTag` parameter of type `uint64` but fails to validate that the value is within the valid XRP destination tag range (0 to $2^{32} - 1$). According to the XRP Ledger specification and the FDC attestation interface documentation (in `IXRPPayment.sol`), "currently XRPL only supports destination tags that are `uint32` values". When a redeemer provides a destination tag value greater than or equal to 2^{32} , the agent cannot create a valid XRP transaction with that tag. This mismatch causes the redemption payment confirmation to fail, where `_payment.destinationTag` (from the FDC proof, stored as `uint256` but constrained to `uint32` range) will never match `request.destinationTag` (stored as `uint64`, potentially $> 2^{32}$). The agent is then forced into redemption default through no fault of their own.

Impact

An agent can be unfairly forced into redemption default when a redeemer (maliciously or accidentally) provides a destination tag value greater than or equal to 2^{32} .

Recommendations

Add validation in `redeemWithTag()` to ensure the destination tag is within the valid XRP range.

```
function redeemWithTag(
    uint256 _amountUBA,
    string memory _redeemerUnderlyingAddressString,
    address payable _executor,
    uint64 _destinationTag
)
    external payable
    notEmergencyPaused
    nonReentrant
```

```
    returns (uint256 _redeemedAmountUBA)
{
    require(RedemptionRequests.redeemWithTagSupported(),
        RedeemWithTagNotSupported());
    uint64 amountAMG = Conversion.convertUBAtoAMG(_amountUBA);
    require(amountAMG >= RedemptionRequests.minimumRedeemAmountAMG(),
        RedemptionTooSmall());
    require(_destinationTag < 2 ** 32, DestinationTagTooBig());
    [...]
}
```

Remediation

This issue has been acknowledged by Flare Network, and fixes were implemented in the following commits:

- [6b6af5c8](#) ↗
- [70bdad9c](#) ↗

3.2. The `_setExecutor` function uses the pending executor instead of effective executor, allowing cooldown bypass

Target	MintingTagManager		
Category	Business Logic	Severity	Medium
Likelihood	High	Impact	Medium

Description

MintingTagManager enforces a cooldown for executor changes. When `setAllowedExecutor` is called, the new executor should only become active after `EXECUTOR_CHANGE_ACTIVE_AFTER_SECONDS` (10 minutes). Until that delay expires, `allowedExecutor()` returns `tag.oldAllowedExecutor`, so the previously effective executor remains authorized to perform direct minting and collect the executor fee.

However, `_setExecutor` always reads `tag.allowedExecutor` when setting `oldAllowedExecutor`:

```
function _setExecutor(uint256 _mintingTag, address _executor) internal {
    TagData storage tag = tagData[_mintingTag];
    address currentAllowedExecutor = tag.allowedExecutor;
    if (currentAllowedExecutor != _executor) {
        uint256 allowedExecutorActiveAfterTs = block.timestamp
        + EXECUTOR_CHANGE_ACTIVE_AFTER_SECONDS;
        tag.oldAllowedExecutor = currentAllowedExecutor;
        tag.allowedExecutor = _executor;
        tag.allowedExecutorActiveAfterTs
        = allowedExecutorActiveAfterTs.toUint64();
        emit AllowedExecutorChangePending(_mintingTag, _executor,
        allowedExecutorActiveAfterTs);
    }
}
```

This is incorrect when an executor change is still pending. In that state, `tag.allowedExecutor` stores the next executor, not the currently effective one.

For example, assume A is the active executor. If the owner first calls `setAllowedExecutor(tag, B)`, then during the cooldown

- `tag.oldAllowedExecutor = A`
- `tag.allowedExecutor = B`

If the owner then calls `setAllowedExecutor(tag, C)` before the first cooldown expires,

`_setExecutor` sets

- `tag.oldAllowedExecutor = B`
- `tag.allowedExecutor = C`
- `tag.allowedExecutorActiveAfterTs = block.timestamp + 10 minutes`

As a result, `allowedExecutor()` returns B during the new cooldown period, even though B was never actually the effective executor. The true previously active executor, A, is immediately displaced.

Impact

This breaks the intended protection of the 10-minute cooldown. A tag owner can bypass the safeguard with two consecutive `setAllowedExecutor` calls, causing the currently active executor to lose authorization before the cooldown has actually protected them.

That matters because the cooldown is designed to protect an executor who may already have paid for an FDC attestation request in anticipation of executing a direct minting. By replacing the effective executor with a never-active pending executor, the owner can lock out the original executor and cause them to lose the FDC attestation fee they already spent.

Recommendations

In `_setExecutor`, derive `oldAllowedExecutor` from the currently effective executor rather than from the raw `tag.allowedExecutor` storage field:

```
address currentAllowedExecutor = tag.allowedExecutor;  
address currentAllowedExecutor =  
    block.timestamp >= tag.allowedExecutorActiveAfterTs  
    ? tag.allowedExecutor  
    : tag.oldAllowedExecutor;
```

This ensures that during a pending cooldown, a second `setAllowedExecutor` call preserves the truly active executor in `oldAllowedExecutor`, maintaining the cooldown guarantee for the executor who may have already initiated an FDC request.

Remediation

This issue has been acknowledged by Flare Network, and a fix was implemented in commit [62842baa](#).

3.3. Use of nonupgradable contract

Target	MintingTagManager		
Category	Coding Mistakes	Severity	Medium
Likelihood	Low	Impact	Medium

Description

The MintingTagManager contract inherits from ERC721Enumerable (the standard OpenZeppelin implementation) while also inheriting from GovernedUUPSProxyImplementation, making it an upgradable contract. The ERC721Enumerable contract uses standard Solidity storage layout, which means its state variables are stored sequentially starting from slot 0. When this contract is upgraded to a new implementation, any changes to the inheritance hierarchy or addition of new state variables in parent contracts could cause storage collisions, where new variables overwrite existing data.

The root cause is that upgradable contracts must use storage-gap patterns or namespaced storage to ensure that storage layouts remain compatible across upgrades. The standard ERC721Enumerable from OpenZeppelin does not follow upgradable storage patterns and lacks storage gaps, making it unsafe for use in UUPS proxies.

Impact

If the contract is upgraded and the new implementation modifies the inheritance order, adds parent contract state variables, or changes the implementation, storage slots could be corrupted. Storage corruption could render the contract unusable, preventing minting tag creation, transfers, or lookups, effectively halting the minting functionality that depends on this contract.

Recommendations

Replace ERC721Enumerable with ERC721EnumerableUpgradeable from OpenZeppelin's upgradeable contracts library.

Remediation

This issue has been acknowledged by Flare Network, and a fix was implemented in commit [c3ecad39](#).

3.4. Unsafe minting function may cause loss of NFTs

Target	MintingTagManager		
Category	Coding Mistakes	Severity	Medium
Likelihood	Low	Impact	Medium

Description

The `_reserve()` function uses `ERC721::_mint()` instead of `ERC721::_safeMint()` when minting NFT minting tags to recipients. When a minting tag is reserved for a smart contract that does not implement the `IERC721Receiver` interface or has no mechanism to transfer the NFT, the minting tag will be locked in that contract. This violates the ERC-721 standard's recommendation to use `ERC721::_safeMint()` for minting operations, which includes a callback to `onERC721Received()` to verify the recipient can handle the token.

OpenZeppelin's [documentation](#) ⁷ discourages `ERC721._mint()` and recommends using `ERC721._safeMint()` whenever possible.

Impact

Users who reserve minting tags from smart contracts (such as multi-sigs, DAOs, or custom treasury contracts) that do not implement the `IERC721Receiver` interface or have no mechanism to transfer the NFT will be locked in that contract.

Recommendations

Use `ERC721._safeMint()` instead to ensure recipient compliance with ERC-721 standards.

Remediation

This issue has been acknowledged by Flare Network, and a fix was implemented in commit [778567b4](#) ⁷.

3.5. Core-vault donation-tag bypass enables grieving attack

Target	CoreVaultClientFacet		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

The `confirmCoreVaultDonation()` function is designed to allow anyone to confirm donation payments made to the core vault's underlying address with the correct destination tag. However, the function does not validate that the payment lacks a payment reference in its memo field. In the FAsset system, payment references are used to uniquely identify payments for specific operations (minting, redemptions, etc.).

If an agent accidentally includes the `coreVaultDonationTag` as the XRP destination tag in a payment that also contains a valid payment reference (intended for another operation like `confirmXRPRedemptionPayment()`), an attacker can front-run the legitimate transaction by calling `confirmCoreVaultDonation()` first. This marks the payment as used, preventing the agent from using the same payment proof for its intended purpose.

The vulnerability exists because the function only checks the destination tag and receiving address – not whether the payment contains a payment reference that would make it ineligible as a donation.

Impact

If an agent accidentally includes the `coreVaultDonationTag` as the XRP destination tag, an attacker can grief agents performing core vault transfers by forcing them to default on their own successful payments. This is the attack flow:

1. An agent calls `transferToCoreVault()` and receives a redemption request with a `REDEMPTION` payment reference.
2. The agent makes the underlying payment to the core vault address with the redemption payment reference in the memo field but mistakenly also includes the `coreVaultDonationTag` as the XRP destination tag.
3. An attacker monitors for such payments and front-runs with `confirmCoreVaultDonation()`.
4. The `CoreVaultManager` marks the transaction ID as confirmed and credits the amount as a donation.

5. The agent's subsequent `confirmXRPRedemptionPayment()` call permanently reverts with `AlreadyConfirmed()`.
6. The agent must wait for the attestation window to expire (minimum one day per `attestationWindowSeconds` setting).
7. The agent calls `finishRedemptionWithoutPayment()` to trigger the default, which charges a penalty (configured via `transferDefaultPenaltyBIPS`) from the agent's vault collateral to the core vault's native address and recreates the redemption tickets, requiring the agent to handle redemptions again.

Recommendations

To prevent any payment from being treated as a donation, consider introducing a dedicated `CORE_VAULT_DONATION` payment reference type that must be used for all donation payments, and update `confirmCoreVaultDonation()` to require the donation payment reference. This ensures donations are explicitly marked and cannot conflict with other payments.

Remediation

This issue has been acknowledged by Flare Network, and a fix was implemented in commit [142ecb4a](#). Payments that include memo data can no longer be used for core vault donations.

3.6. Nonstandard initialization pattern

Target	AssetManagerController		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

The AssetManagerController contract inherits from UUPSUpgradeable but uses a custom initialization mechanism (`GovernedBase::initialise()`) instead of OpenZeppelin's standard Initializable pattern (`initializer / reinitializer` modifiers). The `initialize()` function calls `GovernedBase.initialise()`, which uses a simple boolean flag (`initialised`) to prevent reexecution. This design prevents any reinitialization logic from running during contract upgrades.

Impact

Because the initialized flag is a single boolean with no version counter, there is no way to safely initialize new state variables introduced in future upgraded implementations. Any upgrade requiring postupgrade initialization logic forces each upgrade to create new state variables to prevent reexecution.

Recommendations

Consider refactoring the initialization mechanism to use OpenZeppelin's standard initialization pattern and calling `_disableInitializers()` in the implementation constructor. This maintains backward compatibility while enabling proper reinitialization in future upgrades.

Remediation

This issue has been acknowledged by Flare Network.

Flare Network provided the following response to this finding:

Inheriting from Initializable would change layout of data, so we cannot do it as upgrade. If we need an upgrade with reinitializer, we will add a version field at that time.

3.7. The minimumRedeemAmountAMG is not enforced by lot-based redeem

Target	RedemptionRequestsFacet		
Category	Business Logic	Severity	Low
Likelihood	Low	Impact	Low

Description

The minimumRedeemAmountAMG parameter is intended to prevent redemptions below a governance-defined threshold. That check is enforced in both redeemWithTag and redeemAmount:

```
require(amountAMG >= RedemptionRequests.minimumRedeemAmountAMG(),
    RedemptionTooSmall());
```

However, the lot-based redeem function does not apply the same validation. Its only effective minimum is one lot, since `_lots = 0` reverts with `RedeemZeroLots()`.

At the same time, both the initializer (`_initRedemptionWithTag`) and the governance setter (`setMinimumRedeemAmountUBA`) allow minimumRedeemAmountAMG to be configured up to 10 lots. If the configured minimum exceeds one lot, then `redeem(1, ...)` will still succeed, while `redeemAmount` and `redeemWithTag` for the same one-lot amount will revert with `RedemptionTooSmall()`.

Impact

The configured minimum redemption threshold is not enforced uniformly across redemption paths. If governance sets minimumRedeemAmountAMG above the amount represented by some whole-lot redemption, a user can avoid that threshold by using `redeem` instead of `redeemAmount` or `redeemWithTag`. In effect, minimumRedeemAmountAMG functions as a path-dependent restriction rather than a protocol-wide minimum.

Recommendations

Align the minimum redemption check across all redemption paths. Either

- Cap minimumRedeemAmountAMG at one lot, so it applies only to sub-lot redemptions.
- Enforce minimumRedeemAmountAMG in `redeem`, so all redemption methods respect the same minimum.

Remediation

This issue has been acknowledged by Flare Network.

Flare Network provided the following response to this finding:

A minimum redeem amount is omitted because if it is set lower than the lot size, it has no effect, and if it is set higher, it unexpectedly prevents existing users from redeeming a single lot.

3.8. The convertFromUSD5 function returns unnormalized amounts when tokenFtsoSymbol is empty

Target	Conversion		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

A special-case branch for collateral types whose tokenFtsoSymbol is empty is included in Conversion::convertFromUSD5. As the in-line comment notes, this branch assumes the collateral is a USD-like stablecoin and therefore skips the FTSO lookup, returning the input USD5 amount directly:

```
function convertFromUSD5(
    uint256 _amountUSD5,
    CollateralTypeInt.Data storage _token
)
    internal view
    returns (uint256)
{
    // if tokenFtsoSymbol is empty, it is assumed that the token is a USD-like
    // stablecoin
    // so `_amountUSD5` is (approximately) the correct amount of tokens
    if (bytes(_token.tokenFtsoSymbol).length == 0) {
        return _amountUSD5;
    }
    return convertFromUSD5(_amountUSD5, _token.tokenFtsoSymbol,
        _token.decimals);
}
```

However, this return value is not normalized to the collateral token's decimal precision. As a result, it is only correct when the collateral token itself uses exactly five decimals. For any collateral with decimals != 5, the function returns a value in USD5 precision rather than in the token's base units.

Impact

At the time of writing, this code path is unreachable in production since every deployed collateral type has a nonempty tokenFtsoSymbol.

However, if governance were to add a collateral type in the future with an empty `tokenFtsoSymbol` and `decimals != 5`, any reward flow that relies on `convertFromUSD5` would become mis-scaled. In particular, the challenger rewards would be incorrect and confirmation-by-others rewards would also be paid in an incorrect amount.

Recommendations

Normalize the USD5 amount to the collateral token's decimal precision in the empty-symbol branch so that the function always returns token base units:

```
if (bytes(_token.tokenFtsoSymbol).length == 0) {  
    return _amountUSD5;  
    return _amountUSD5.mulDiv(10 ** _token.decimals, 1e5);  
}
```

Remediation

This issue has been acknowledged by Flare Network, and a fix was implemented in commit [e97c739f](#).

3.9. By counting nondistributable fees, enter inflates the zero-supply entry requirement

Target	CollateralPool		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

When the pool token supply falls to zero, `CollateralPool::enter` enforces a minimum collateral deposit proportional to `totalFAssetFees`:

```
if (totalPoolTokens == 0) {
    // if the pool is empty of tokens and collateral but contains fees, the new
    // entrant can claim them,
    // but must add enough collateral so that ratio fees/tokens stays normal
    // for future fees
    AssetPrice memory assetPrice = _getAssetPrice();
    require(msg.value >= totalFAssetFees.mulDiv(assetPrice.mul,
        assetPrice.div), AmountOfCollateralTooLow());
}
```

This check uses `totalFAssetFees`, which represents the gross fee balance, rather than the amount of fees that is actually distributable, as returned by `_totalVirtualFees()`.

If an LP exits without calling `withdrawFees`, negative fee debt can remain in the pool. In that case, `totalFAssetFees` may stay positive even though the distributable fees are zero or materially lower.

As a result, after the last LP exits and the pool token supply becomes zero, a subsequent `enter()` call may require the next entrant to post collateral against the full gross fee balance `totalFAssetFees`, even though those fees are not actually distributable. This creates an unnecessary barrier to reentering the pool.

Impact

A malicious final LP can exit all tokens while leaving fees unwithdrawn (creating negative fee debt). This inflates the requirement for a user to reenter the pool.

Recommendations

Use `_totalVirtualFees()` instead of `totalFAssetFees` in the zero-supply collateral check so the required deposit is based only on distributable fees:

```
require(msg.value >= totalFAssetFees.mulDiv(assetPrice.mul, assetPrice.div),
    AmountOfCollateralTooLow());
uint256 virtualFees = _totalVirtualFees();
require(msg.value >= virtualFees.mulDiv(assetPrice.mul, assetPrice.div),
    AmountOfCollateralTooLow());
```

Remediation

This issue has been acknowledged by Flare Network, and a fix was implemented in commit [084b5ccb](#).

3.10. Delayed direct mints do not reserve minting cap capacity

Target	DirectMintingFacet		
Category	Business Logic	Severity	Low
Likelihood	Low	Impact	Low

Description

Commit [62b43f70](#) introduced a minting cap check in `executeDirectMinting` before the rate limit check that may delay a direct mint:

```
// check minting cap before rate limits, to prevent delayed mintings from
// piling up while waiting
// for minting capacity to increase
Minting.checkMintingCap(Conversion.convertUBAToAmg(receivedAmount));
// check rate limits
DirectMintingDelayState mintingDelayed =
    _checkRateLimits(_payment.data.requestBody.transactionId, receivedAmount);
```

However, if the mint is delayed, the function returns before minting FAssets. The delayed minting state stores only timestamps:

```
struct DelayedMinting {
    uint64 startedAt;
    uint64 allowedAt;
}
```

Therefore, delayed direct mints are not reserved against the cap. On release, `executeDirectMinting` is called again and performs the same cap check before recognizing that the mint was already delayed and is now releasable. If cap capacity was consumed in the meantime, release reverts with `MintingCapExceeded`.

Impact

Multiple delayed direct mints can be queued even if their combined amount exceeds the available minting cap. More importantly, a user can send underlying funds to the core vault, successfully queue a delayed direct mint, and later be unable to mint the corresponding FAssets because other minting consumed the cap during the delay window. The user's mint remains un-releasable until cap capacity becomes available again or governance changes the cap.

Recommendations

Reserve the queued amount against the cap when a delayed mint is queued, and skip the cap check when releasing an already-queued delayed direct mint.

Remediation

This issue has been acknowledged by Flare Network, and a fix was implemented in commit [e3ded6b5](#) ↗.

3.11. Inconsistency in DirectMintingPaymentTooSmallForFee emission

Target	MintingTagManager		
Category	Coding Mistakes	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The `executeDirectMinting()` function emits the `DirectMintingPaymentTooSmallForFee` event with `state.minimumMintingFeeAmt` for the `mintingFeeUBA` parameter, which is inconsistent with the parameter name.

```
event DirectMintingPaymentTooSmallForFee(
    bytes32 transactionId,
    uint256 receivedAmount,
    uint256 mintingFeeUBA);
```

```
function executeDirectMinting(
    IXRPPayment.Proof calldata _payment
)
    external payable
    onlyAttached
    notEmergencyPaused
    nonReentrant
{
    [...]
    if (paymentTooSmall) {
        // If the total payment is less than the system fee, everything goes to
        // the fee receiver and no further
        // actions are done, to prevent smart accounts users from sending very
        // small amounts to avoid paying fee.
        // Executor also gets nothing in this case since the minting fee has
        // priority over the executor fee.
        require(msg.value == 0, NoValueExpected());
        emit
        DirectMintingPaymentTooSmallForFee(_payment.data.requestBody.transactionId,
        receivedAmount, Conversion.convertAmtToUBA(state.minimumMintingFeeA
        mg));
    } else if (mintToSmartAccount) {
```

```
        _mintToSmartAccounts(_payment, receivedAmount, mintingFeeUBA);
    } else {
        require(msg.value == 0, NoValueExpected());
        _mintToRecipient(_payment, recipient, receivedAmount, mintingFeeUBA,
            executorFeeUBA);
    }
}
```

Impact

Inconsistent event emission can lead to inaccurate off-chain tracking, causing confusion or faulty data processing.

Recommendations

Update the `DirectMintingPaymentTooSmallForFee` event and its emission to be consistent.

Remediation

This issue has been acknowledged by Flare Network, and a fix was implemented in commit [d110dd0d](#).

3.12. Direct minting limiter behaves as a throttling mechanism rather than a strict per-window cap

Target	MintingRateLimiter		
Category	Business Logic	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The MintingRateLimiter used by DirectMintingFacet for hourly and daily direct minting does not enforce a hard cap on the amount that can actually be minted within a given hour or day. Instead, it meters minting when requests are submitted, delays excess volume in proportion to accumulated backlog, and lets that backlog decay by one window's worth of capacity per elapsed window. In effect, the mechanism behaves as a throttling control rather than a hard per-window execution cap.

This distinction may be easy to overlook based on the documentation as of the time of writing. The specification for DirectMinting says that "the total minted amount is limited on hourly and daily basis", and the directMintingHourlyLimitUBA and directMintingDailyLimitUBA settings likewise imply strict per-window caps. The implementation, however, provides pacing rather than a hard execution-time ceiling.

Impact

This does not let an attacker bypass the configured delay or accelerate release of delayed mintings. However, operators and integrators should not interpret the hourly and daily direct minting limits as strict execution-time caps. The control provides pacing and review time for excess minting requests, but once delayed mintings are released, they are no longer capped by the current hourly or daily window and may execute alongside newly admitted minting.

Recommendations

Clarify the specification and setting descriptions to state that the hourly and daily direct minting controls are throttling mechanisms, not strict execution-time caps. If a strict execution-time cap is desired, delayed mintings would need to be reaccounted when they are executed.

Remediation

This issue has been acknowledged by Flare Network, and a fix was implemented in commit [67c2166b](#). The documentation for DirectMinting has been updated accordingly.

3.13. Missing character validation allows invalid underlying addresses in redemption requests

Target	RedemptionRequests		
Category	Coding Mistakes	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

Note: This finding was discovered independently by Flare Network.

The `createRedemptionRequest()` function accepts a `redeemerUnderlyingAddressString` parameter that represents the destination address on the underlying chains. While the function validates the address length, it does not validate that the string contains only valid ASCII characters or the underlying chain's address format.

Impact

This issue does not materially impact agents, as they can invoke `rejectInvalidRedemption` to discharge the redemption obligation.

The impact is primarily on users who unintentionally submit redemption requests with invalid underlying addresses, such as addresses that contain hidden or non-printable characters introduced during copying and pasting. In such cases, users cannot receive the redemption payout even though they have already burned the fassets.

Recommendations

Add validation to ensure the `redeemerUnderlyingAddressString` contains only printable ASCII characters, so format errors are caught before redemption requests are created.

Remediation

This issue has been acknowledged by Flare Network, and a fix was implemented in commit [cefd1b31](#).

4. Patch Review

The purpose of this section is to document the exact commit diff of the codebase that was considered in scope for this audit.

As requested by Flare Network, we focused on the changes made between commit [4c52db64](#) and the commit [d54b04c8](#).

4.1. Introduction of the direct minting system

The update introduced several new contracts and libraries to support direct minting functionality, allowing users to mint FAssets by sending underlying assets (e.g., XRP) directly to the core vault address.

Contract: DirectMintingFacet

The DirectMintingFacet contract enables direct minting of FAssets through underlying asset payments to the core vault. It contains the following functions:

- It contains `executeDirectMinting`, which processes direct minting payments by verifying XRP payment proofs, validating tags and memo data, checking rate limits, and minting FAssets to recipients or smart accounts. The function handles three scenarios: payments too small for fees (all goes to fee receiver), minting to smart accounts (notifies smart account manager), and minting to regular recipients (distributes fees to system and executor).
- It contains `markUnblockedDirectMintingAllowed`, which allows marking delayed mints as executable when they have been unblocked by governance, ensuring the exclusive executor period begins from the unblock time.
- It contains `directMintingPaymentAddress`, which returns the core vault underlying address where users should send payments for direct minting.
- It contains `directMintingDelayState`, which returns the delay state, allowed execution time, and start time for a specific transaction.

Contract: DirectMintingSettingsFacet

The DirectMintingSettingsFacet contract manages governance-controlled settings for direct minting. It contains the following functions:

- The `unblockDirectMintingsUntil` function, which allows immediate governance to unblock all delayed minting that started before a specified timestamp (must be in the past to ensure manual validation)
- The `setMintingTagManager` function, which sets the minting tag manager contract address (cannot be zero)
- The `setSmartAccountManager` function, which sets the smart account manager contract address (cannot be zero)

- The `setDirectMintingFeeReceiver` function, which sets the address that receives minting fees
- The `setDirectMintingFee` function, which updates the minting fee percentage (in bips) and minimum fee amount (in UBA), with validation that changes cannot exceed 4x increase or 1/4 decrease
- The `setDirectMintingExecutorFee` function, which updates the executor fee for processing direct minting
- The `setDirectMintingOthersCanExecuteAfterSeconds` function, which sets the time after which anyone can execute a direct minting (maximum one day)
- The `setDirectMintingHourlyLimitUBA` and `setDirectMintingDailyLimitUBA` functions, which update the hourly and daily minting limits, respectively
- The `setDirectMintingLargeMintingThrottling` function, which configures the threshold and delay for large minting (maximum delay of three days)
- Multiple getter functions for retrieving current settings and limiter states

Contract: MintingTagManager

The `MintingTagManager` contract is an ERC-721–based system for managing destination tags used in direct minting. It inherits from `ERC721Enumerable` and `GovernedUUPSProxyImplementation`. It contains the following:

- The `reserve` function, which allows users to reserve a unique minting tag by paying a reservation fee (with reentrancy protection)
- The `setMintingRecipient` function, which sets the recipient address for minted FAssets for a specific tag (only the tag owner can call)
- The `setAllowedExecutor` function, which sets an executor address that has exclusive rights to execute mintings with a specific tag (only the tag owner can call) – changes have a 10-minute cooldown period
- The `transfer` function, which transfers tag ownership to another address
- The `reservedTagsForOwner` function, which returns all tag IDs owned by a specific address
- The `mintingRecipient` function, which returns the current recipient address for a tag
- The `allowedExecutor` function, which returns the current allowed executor for a tag, respecting the cooldown period for executor changes
- The `pendingAllowedExecutorChange` function, which returns information about pending executor changes that have not become active yet
- The `setReservationFee` and `setReservationFeeRecipient` functions, which allow governance to update reservation parameters

Extended redemption interfaces and facets

The `IRedeemExtended` interface defines extended redemption functionality:

- The `redeemAmount` function, which allows redeeming an arbitrary amount in UBA instead of whole lots
- The `redeemWithTag` function, which allows redeeming with a required XRP destination tag
- The `confirmXRPRedemptionPayment` function, which confirms redemption payments that include destination tags using `IXRPPayment` proofs
- The `xrpRedemptionPaymentDefault` function, which handles defaults for XRP redemptions with tags using `IXRPPaymentNonexistence` proofs
- The `redeemWithTagSupported` function, which indicates whether the chain supports destination tags

The **RedeemExtendedSettingsFacet** manages settings:

- The `setMinimumRedeemAmountUBA` function, which sets the minimum amount for UBA-based redemptions (max 10 lots, maximum 4x increase)
- The `minimumRedeemAmountUBA` function, which returns the current minimum redemption amount

Initialization contract

The **DirectMintingAndRedeemExtendedInit** contract initializes new features during diamond upgrade:

- The `initialize` feature, which sets up direct minting parameters (tag manager, smart account manager, fees, rate limiters, throttling), redemption with tag settings (support flag, minimum amount), and core vault client settings (donation tag)
- Updates the diamond's supported interfaces to include `IDirectMinting`, `IDirectMintingSettings`, `IRedeemExtended`, `IRedeemExtendedSettings`, and updated `ICoreVaultClient` interfaces

4.2. Changes in contract structure

These changes affect several contracts and libraries across the system.

Redemption flow refactoring

The redemption system was significantly refactored to support multiple redemption methods and XRP destination tags:

- The `redeem` function in **RedemptionRequestsFacet** now internally calls a unified `_redeem` helper function that accepts amount in AMG and a granularity parameter.
- A new internal `_redeem` function consolidates redemption logic for lot-based, UBA-based, and tag-based redemptions.

- The `_redeemFirstTicket` helper function logic was updated to support the new unified flow.
- All redemption-request-creation calls now use the structured `CreateRedemptionRequestArgs` format instead of passing 11+ individual parameters.

Payment confirmation refactoring

The payment confirmation system was refactored to support multiple proof types:

- **RedemptionConfirmationsFacet** now has two confirmation entry points: `confirmRedemptionPayment` (for `IPayment` proofs) and `confirmXRPPayment` (for `IXRPPayment` proofs).
- Both entry points extract payment data into a `RedemptionPaymentData` struct and call a unified `_confirmRedemptionPayment` internal function.
- The `_validatePayment` function was refactored to accept `RedemptionPaymentData` instead of a proof, allowing validation logic to be shared across proof types.
- Destination tag validation was added; if a redemption requires a destination tag, the proof type must support tags, and the tag value must match.
- The memo-data-to-payment-reference conversion logic was extracted to support XRP proofs.

Core vault integration refactoring

The core vault integration was refactored to separate proof verification from payment confirmation:

- **CoreVaultManager**'s `confirmPayment` function no longer accepts or verifies proofs directly.
- Proof verification is now done in the `AssetManager` facets before calling `confirmPayment`.
- The function signature changed from accepting `IPayment.Proof` to accepting `transactionId` and `receivedAmount` directly.
- The `onlyAssetManager` modifier was added to restrict calls to the registered `AssetManager`.
- The **CoreVaultClient** library was updated with a new `confirmCoreVaultPayment` helper that wraps the manager call.

Payment confirmation recording refactoring

The **PaymentConfirmations** library was refactored to support multiple proof types:

- A new `confirmIncomingPayment` overload was added for `IXRPPayment.Proof` to record direct minting payments.
- A new `confirmSourceDecreasingTransaction` overload was added that accepts transaction ID and source address hash directly.

- This allows confirmation recording without passing the full proof object through multiple layers.

Attestation verification refactoring

The **TransactionAttestation** library was expanded to support XRP-specific proofs:

- New verification functions `verifyXRPPayment`, `verifyXRPPaymentSuccess`, and `verifyXRPPaymentNonexistence` were added.
- A new `verifyProofOwnership` function was added to validate the `proofOwner` field in FDC proofs.
- Import statements were refactored to use specific interface imports instead of the combined `IFdcVerification` import.

Data structure refactoring

The data structures were refactored to support direct minting.

The **Redemption.Request** struct changes:

- Two new fields were added: `requiresDestinationTag` (bool) and `destinationTag` (uint64).
- These fields enable redemption requests to specify when a destination tag is required in the payment.

The **Agent.State** struct changes:

- The `_endMarker` placeholder field was replaced with `activeReturnFromCoreVaultStartTs` (uint64).
- This field tracks when a return from core vault was initiated, enabling "confirmation by others" after a time-out.

The **PaymentReference** library changes:

- New payment reference types were added: `DIRECT_MINTING` (for standard direct minting) and `DIRECT_MINTING_EX` (for extended format with executor).
- Helper functions were added to encode and decode these new reference types.

Settings management refactoring

Settings management was refactored.

SettingsManagementFacet changes:

- New governance methods were added to manage direct minting settings through the `AssetManagerController` timelock.

- Methods include setting the minting tag manager, smart account manager, fees, limits, and throttling parameters.
- All new methods use the existing governance and rate-limiting patterns.

CoreVaultClientSettingsFacet changes:

- A new `setCoreVaultDonationTag` function was added to allow governance to configure the donation tag.
- A `getCoreVaultDonationTag` getter function was added.

Conversion and math refactoring

Conversion and math were refactored.

Conversion library changes:

- Additional conversion functions were added to support UBA-based operations throughout the system.
- Conversion logic was enhanced to handle edge cases in direct minting fee calculations.

5. Assessment Results

During our assessment on the scoped Flare FAssets V1.3 contracts, we discovered 13 findings. No critical issues were found. One finding was of high impact, three were of medium impact, six were of low impact, and the remaining findings were informational in nature.

5.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.